

Mathematics Foundations

Trinetra-AI — Phase 1 Notes

Linear Algebra · Calculus · Probability · Statistics

Every concept mapped to where it appears in multi-sensor fusion & navigation

These notes cover only the math a sensor-fusion / navigation project actually leans on. They are not a full math course; they are the load-bearing subset, each tied to the Kalman filter, the state vector, or the sensor-error model you will build.

1 Linear Algebra

Linear algebra is the language of state estimation. The thing you are estimating (position, velocity, orientation) is a *vector*; the models that move it forward and the sensors that observe it are *matrices*; and the uncertainty you track is a *covariance matrix*. Get comfortable here and 70% of the Kalman filter stops being mysterious.

1.1 Vectors

A vector is an ordered list of numbers. In navigation it is the **state** you track:

$$\mathbf{x} = [p_x \ p_y \ p_z \ v_x \ v_y \ v_z]^T \in \mathbb{R}^6,$$

here position and velocity in 3D. Sensor readings (a 3-axis accelerometer output, a magnetometer vector) are also vectors.

Core operations.

- **Addition / scaling:** $\mathbf{a} + \mathbf{b}$, $c\mathbf{a}$ — combine and rescale states or measurements.
- **Dot product:** $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$. Measures alignment; used to project one vector onto another and to compute angles between headings.
- **Norm (length):** $\|\mathbf{a}\| = \sqrt{\mathbf{a} \cdot \mathbf{a}} = \sqrt{\sum_i a_i^2}$. This is the Euclidean (L_2) norm — it is literally how “position error in metres” is computed.
- **Unit vector:** $\hat{\mathbf{a}} = \mathbf{a} / \|\mathbf{a}\|$ — pure direction, magnitude 1. Headings and axis-of-rotation are unit vectors.
- **Cross product:** $\mathbf{a} \times \mathbf{b}$ — gives a vector perpendicular to both. Appears in rigid-body rotation and in relating angular velocity (gyro) to how frames turn.

The navigation *state* is a vector, and so is every measurement. The gyroscope gives angular velocity; the accelerometer gives specific force; the magnetometer gives a 3D field vector. Fusing them means doing vector arithmetic every timestep. “Position error (RMSE)” in your evaluation section is just the norm of the difference between your estimated position vector and ground truth.

1.2 Matrices

A matrix is a rectangular grid of numbers, $\mathbf{A} \in \mathbb{R}^{m \times n}$. It represents a *linear transformation* that maps one vector to another: $\mathbf{y} = \mathbf{A}\mathbf{x}$.

Core operations.

- **Multiplication:** $(\mathbf{AB})_{ij} = \sum_k A_{ik}B_{kj}$. Order matters: $\mathbf{AB} \neq \mathbf{BA}$ in general. Inner dimensions must match.
- **Transpose:** $\mathbf{A}^\top$ flips rows and columns. Note $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$.
- **Identity:** $\mathbf{I}$, the “do nothing” matrix: $\mathbf{AI} = \mathbf{A}$.
- **Inverse:** $\mathbf{A}^{-1}$ undoes $\mathbf{A}$, so $\mathbf{AA}^{-1} = \mathbf{I}$. Exists only if $\det(\mathbf{A}) \neq 0$. Solving $\mathbf{Ax} = \mathbf{b}$ gives $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$.

Special matrices you will meet constantly.

- **Symmetric:** $\mathbf{A} = \mathbf{A}^\top$. All covariance matrices are symmetric.
- **Positive (semi-)definite:** $\mathbf{z}^\top \mathbf{Az} > 0$ for all $\mathbf{z} \neq \mathbf{0}$. A valid covariance matrix must be positive semi-definite — negative variance is meaningless.
- **Orthogonal / rotation:** $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$ and $\det(\mathbf{R}) = +1$. These are exactly the **rotation matrices** $\in SO(3)$ that convert a vector between coordinate frames (body $\leftrightarrow$ navigation). Their inverse is just their transpose, which is cheap and numerically clean.

The Kalman filter is written almost entirely in matrices: $\mathbf{F}$ (state transition, “how the state evolves”), $\mathbf{H}$ (measurement model, “what the sensor sees”), and the covariances $\mathbf{P}$, $\mathbf{Q}$, $\mathbf{R}$. Converting a magnetometer reading from the body frame into the navigation frame is a *rotation-matrix* multiply. The Kalman *gain* requires a matrix *inverse*. If you are shaky on matrix multiply, transpose, and inverse, the filter equations will look like noise.

1.3 Eigenvalues & Eigenvectors

For a square matrix $\mathbf{A}$, a nonzero vector $\mathbf{v}$ is an **eigenvector** with **eigenvalue** λ if

$$\mathbf{Av} = \lambda\mathbf{v}.$$

In words: $\mathbf{A}$ acting on $\mathbf{v}$ only stretches it (by λ), never rotates it. The eigenvalues solve the characteristic equation $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$.

Two facts that carry the load:

- A symmetric matrix has *real* eigenvalues and *orthogonal* eigenvectors.
- A matrix is positive definite $\iff$ all its eigenvalues are > 0 .

A covariance matrix $\mathbf{P}$ describes an **uncertainty ellipsoid**. Its *eigenvectors* are the axis directions of that ellipsoid, and the square roots of its *eigenvalues* are the axis lengths (1σ spread). A long thin ellipsoid means you know one direction well and another poorly.

Eigenvalues let you *read* your uncertainty: plotting the covariance ellipse of $\mathbf{P}$ shows where the estimate is confident vs. vague. They also tell you about **stability** — the eigenvalues of the transition matrix $\mathbf{F}$ govern whether errors shrink or blow up over time — and about **numerical conditioning**, which is why a filter can quietly diverge. All eigenvalues of $\mathbf{P}$ staying positive is the sanity check that your covariance is still a legal covariance.

2 Calculus

Motion is change over time, and change is what derivatives measure. Calculus shows up in two distinct places in this project: (1) the *motion model* that propagates the state is a set of differential equations, and (2) the *Jacobian* — a matrix of partial derivatives — is the single idea that turns a Kalman filter into an *Extended* Kalman filter.

2.1 Derivatives

The derivative of $f(t)$ is its instantaneous rate of change:

$$f'(t) = \frac{df}{dt} = \lim_{\Delta t \rightarrow 0} \frac{f(t + \Delta t) - f(t)}{\Delta t}.$$

Rules you will reuse: linearity, the product rule, and above all the **chain rule** $\frac{d}{dt}f(g(t)) = f'(g(t))g'(t)$.

In navigation the state obeys a continuous-time dynamics equation

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}), \quad \text{e.g.} \quad \dot{\mathbf{p}} = \mathbf{v}, \quad \dot{\mathbf{v}} = \mathbf{a},$$

i.e. velocity is the derivative of position and acceleration is the derivative of velocity. Propagating the state forward one timestep is (numerically) integrating this.

Your dead-reckoning / INS mechanization *is* integration of the motion model: the accelerometer feeds $\mathbf{a}$, you integrate to get $\mathbf{v}$, integrate again for $\mathbf{p}$. Understanding derivatives is what makes “the gyro measures the rate of change of orientation” a precise statement rather than a slogan — and it explains why small biases integrate into large *drift*.

2.2 Partial Derivatives & the Jacobian

When a function has several inputs, a **partial derivative** $\partial f / \partial x_i$ measures sensitivity to one input while holding the others fixed. Stacking all partials of a vector-valued function

$f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ gives the **Jacobian** matrix:

$$\mathbf{J} = \frac{\partial f}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

The Jacobian is the *best linear approximation* of a nonlinear function at a point — it is the multivariable generalisation of “slope”.

The EKF works by **linearising** the nonlinear motion model f and measurement model h at the current estimate, using their Jacobians:

$$\mathbf{F} = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}}, \quad \mathbf{H} = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}}.$$

Those $\mathbf{F}$ and $\mathbf{H}$ are then dropped straight into the ordinary Kalman equations. *The “Extended” in EKF is exactly this Jacobian step.*

Real navigation is nonlinear (orientation, magnetic-field lookups, range measurements), so the plain Kalman filter does not apply directly. The EKF — the workhorse you will implement — depends entirely on computing Jacobians of f and h . If you cannot form a Jacobian by hand for a small system, you cannot debug an EKF. (The UKF sidesteps Jacobians with sample points, which is one reason it is often preferred, but you still need this concept to understand *why*.)

2.3 Gradients & Optimisation

For a scalar function $L(\boldsymbol{\theta})$ of many variables, the **gradient** is the vector of partials:

$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial \theta_1} & \cdots & \frac{\partial L}{\partial \theta_n} \end{bmatrix}^\top.$$

It points in the direction of steepest increase, so to *minimise* L you step against it — **gradient descent**:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla L(\boldsymbol{\theta}),$$

with learning rate α .

This is how every ML module in the project trains — the LSTM drift-predictor, the denoising autoencoder, the anomaly detector all learn by gradient descent on a loss function (back-propagation is just the chain rule applied across a network). Gradients also underlie the least-squares fitting used when calibrating sensors. So the same idea, gradients, serves both the classical and the AI halves of Trinetra.

3 Probability

Every sensor lies a little, every timestep. Probability is how you reason with quantities you can only know approximately. The central object is the **Gaussian**, and the central rule is **Bayes' theorem** — together they *are* the Kalman filter.

3.1 Random Variables, Expectation, Variance

A random variable X takes values with a probability density $p(x)$. Two summaries matter most:

$$\text{mean (expectation): } \mu = \mathbb{E}[X], \quad \text{variance: } \sigma^2 = \mathbb{E}[(X - \mu)^2].$$

The standard deviation σ is the typical spread in the same units as X — for a sensor, this is its noise level.

3.2 The Gaussian (Normal) Distribution

The single most important distribution in estimation:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right), \quad X \sim \mathcal{N}(\mu, \sigma^2).$$

Its multivariable version, for a state vector $\mathbf{x} \in \mathbb{R}^n$, uses a mean vector $\boldsymbol{\mu}$ and a **covariance matrix** $\boldsymbol{\Sigma}$:

$$p(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^n \det \boldsymbol{\Sigma}}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right).$$

The quantity $(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})$ is the squared **Mahalanobis distance** — distance measured in units of standard deviations, accounting for correlations. (Notice it needs a matrix *inverse* and a *covariance* — this is where the linear-algebra section pays off.)

The Kalman filter represents the entire state estimate as *one* multivariate Gaussian: a mean $\hat{\mathbf{x}}$ (best guess) and a covariance $\mathbf{P}$ (uncertainty). The whole filter is just: keep this Gaussian, move it forward through the motion model, and shrink it with each measurement. Mahalanobis distance is also how you gate *anomalies* — a measurement too many σ away from prediction is flagged as a fault, which is exactly your fault-detection module.

3.3 Bayes' Theorem

$$\underbrace{p(\mathbf{x} | \mathbf{z})}_{\text{posterior}} = \frac{\overbrace{p(\mathbf{z} | \mathbf{x})}^{\text{likelihood}} \overbrace{p(\mathbf{x})}^{\text{prior}}}{\underbrace{p(\mathbf{z})}_{\text{evidence}}} \propto p(\mathbf{z} | \mathbf{x}) p(\mathbf{x}).$$

Read it as: *updated belief* $\propto$ *how well the measurement fits* $\times$ *what you believed before*.

Every recursive filter (Kalman, EKF, UKF, particle) is Bayes' theorem applied once per timestep:

- **Predict** = push the *prior* through the motion model (uncertainty grows).
- **Update** = fold in the measurement *likelihood* to get the *posterior* (uncertainty shrinks).

For linear systems with Gaussian noise, this Bayesian update has a closed form — and that closed form *is* the Kalman filter.

This is the conceptual backbone of the entire fusion stack. The Kalman filter is not an arbitrary recipe; it is the exact Bayesian answer under linear-Gaussian assumptions. The particle filter is the same Bayesian idea when those assumptions break (multi-modal position, magnetic-map ambiguity), approximated with a swarm of samples. Understand predict/update as prior/posterior and every filter in the project becomes one family instead of four disconnected algorithms.

4 Statistics

Probability tells you how randomness behaves in theory; statistics is how you *measure* it from real data. For this project statistics does two jobs: characterising sensor noise (which fills in the filter's **Q** and **R** matrices) and evaluating how well the system performs.

4.1 Estimating from Samples

From N data points you estimate the true mean and variance:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i, \quad s^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2.$$

The $N - 1$ (Bessel's correction) makes s^2 an unbiased estimate of the variance.

4.2 Covariance & Correlation

For two quantities, covariance measures how they move together:

$$\text{cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)], \quad \rho_{XY} = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y} \in [-1, 1].$$

Stacked for a whole state vector, this gives the **covariance matrix P**: diagonal entries are per-variable variances, off-diagonals encode correlations (e.g. how position error couples to velocity error).

4.3 Noise Models for Sensors

Real sensor error is a sum of distinct effects, each characterised statistically:

- **White noise**: zero-mean, uncorrelated in time. Sets the measurement-noise covariance **R**.
- **Bias**: a slowly-varying offset (nonzero mean). If ignored, it integrates into drift.

- **Random walk:** the integral of white noise — its variance *grows with time*, which is exactly why unaided IMU position drifts unboundedly.

The **Allan variance** is the standard tool that separates these components from a long static recording, and is how you will justify the numbers you put into **Q** and **R**.

4.4 Evaluation Metrics

To report results you need error statistics. The main one:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N \|\hat{\mathbf{p}}_i - \mathbf{p}_i^{\text{true}}\|^2},$$

the root-mean-square position error against ground truth. Related figures: mean orientation error (degrees) and *drift rate* (error per distance travelled), which is the honest way to report a GPS-denied system's performance.

The filter's tuning matrices **Q** (process noise) and **R** (measurement noise) are *statistical statements* about your sensors — you get their values by characterising noise, not by guessing. Off-diagonal covariance terms are what let the filter exploit correlations between states. And every claim in your evaluation section (“X% lower drift than a plain EKF”) is an RMSE / drift-rate comparison. Statistics is both the input (tuning) and the output (results) of the whole system.

5 How It All Comes Together: One Kalman Step

Here is the standard (linear) Kalman filter, annotated so you can see all four topics doing their job in a single timestep. This is the payoff — it is *why* these four areas were the ones to learn.

Predict (push the belief forward):

$$\hat{\mathbf{x}}^- = \mathbf{F}\hat{\mathbf{x}} + \mathbf{B}\mathbf{u}, \quad \mathbf{P}^- = \mathbf{F}\mathbf{P}\mathbf{F}^\top + \mathbf{Q}.$$

Update (fold in the measurement **z**):

$$\mathbf{K} = \mathbf{P}^- \mathbf{H}^\top (\mathbf{H}\mathbf{P}^- \mathbf{H}^\top + \mathbf{R})^{-1},$$

$$\hat{\mathbf{x}} = \hat{\mathbf{x}}^- + \mathbf{K} (\mathbf{z} - \mathbf{H}\hat{\mathbf{x}}^-), \quad \mathbf{P} = (\mathbf{I} - \mathbf{K}\mathbf{H}) \mathbf{P}^-.$$

Element	Which math it comes from
State $\hat{\mathbf{x}}$, measurement $\mathbf{z}$	Vectors (Linear Algebra)
$\mathbf{F}, \mathbf{H}, \mathbf{B}$, all matrix products & transposes	Matrices (Linear Algebra)
The inverse $(\cdot)^{-1}$ in the gain $\mathbf{K}$	Matrix inverse (Linear Algebra)
$\mathbf{F}, \mathbf{H}$ as Jacobians (EKF case)	Partial derivatives (Calculus)
$\mathbf{P}, \mathbf{Q}, \mathbf{R}$ as covariances	Gaussians / covariance (Probability)
Predict $\rightarrow$ Update as prior $\rightarrow$ posterior	Bayes' theorem (Probability)
Values inside $\mathbf{Q}, \mathbf{R}$; RMSE evaluation	Noise statistics (Statistics)
Positive-definiteness / stability of $\mathbf{P}, \mathbf{F}$	Eigenvalues (Linear Algebra)

Master these four topics against this one diagram, and the $KF \rightarrow EKF \rightarrow UKF \rightarrow \text{Particle Filter}$ progression becomes variations on a theme rather than four things to memorise.